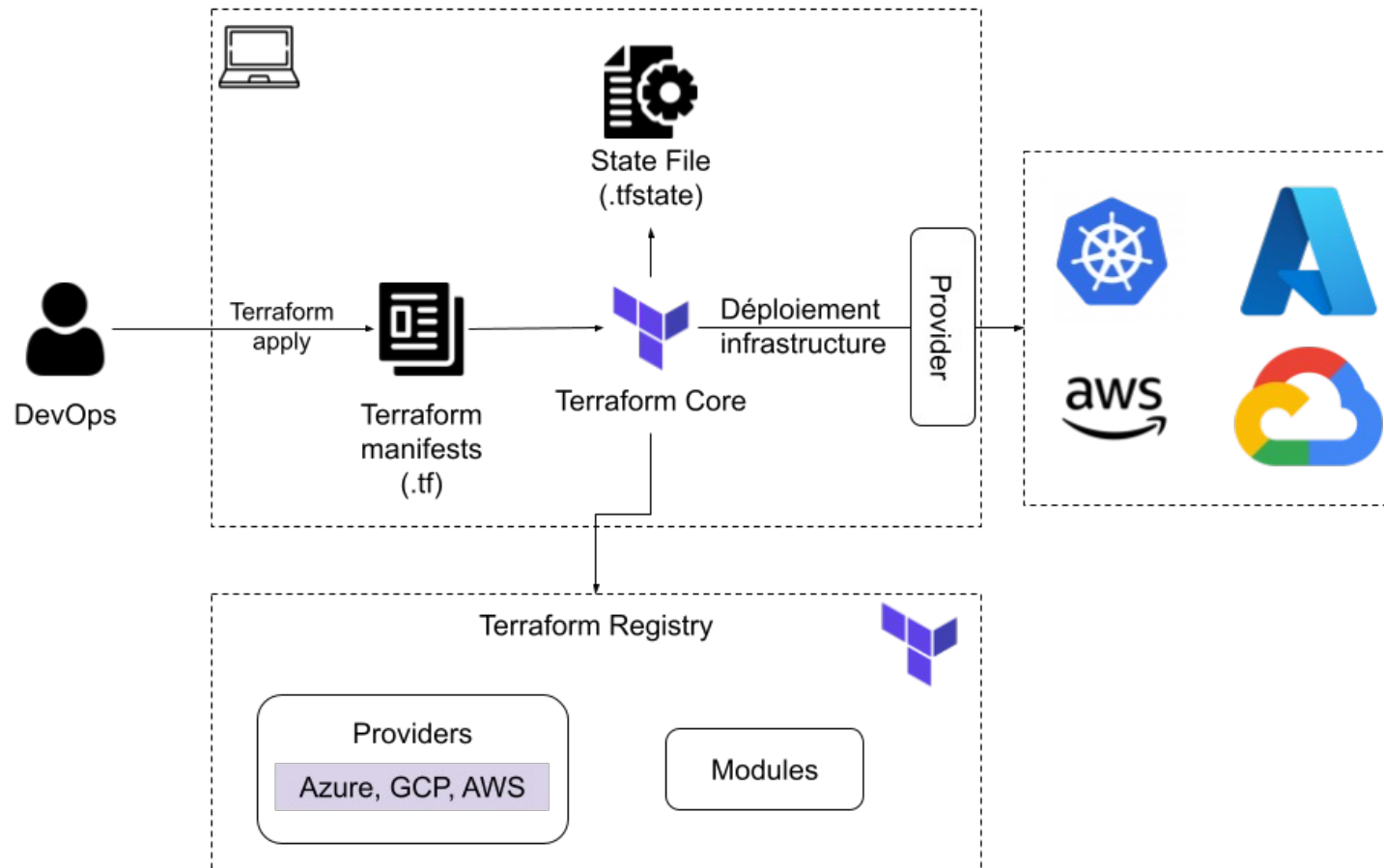




HashiCorp
Terraform

Architecture Terraform



Architecture Terraform



Terraform Core

Le composant principal de **Terraform Core** est un CLI (pour *Command Line Interface* ou interface en ligne de commandes). C'est avec ce composant que les utilisateurs interagissent directement, au travers de commandes exécutées dans un *shell*.



Terraform
manifests
(.tf)

La **configuration** (manifest) que vous allez déployer sera contenue dans un ou plusieurs fichiers Terraform dont l'extension est **.tf**.



State File
(.tfstate)

Le state peut être vu comme une **base de données**, qui contient **l'état des ressources qui ont été créées** par Terraform et leur lien avec le code.

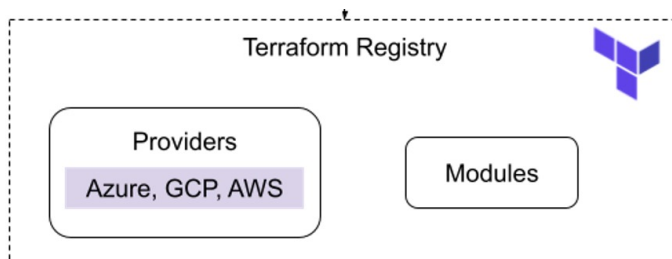
Architecture Terraform



State File
(.tfstate)

Lors de son exécution, Terraform compare le **code** au **state**, calcule la différence entre les deux, et planifie les opérations à exécuter :

- une ressource présente dans le code, mais absente du state doit être créée ;
- une ressource présente dans le state, mais absente du code doit être supprimée ;
- si une ressource est présente dans le state et dans le code, et que des attributs sont modifiés, elle doit être mise à jour, ou recrée si la mise à jour n'est pas possible.



<https://registry.terraform.io/>

Le registre héberge l'ensemble des providers, ainsi que les modules, qui sont des éléments de code réutilisables.

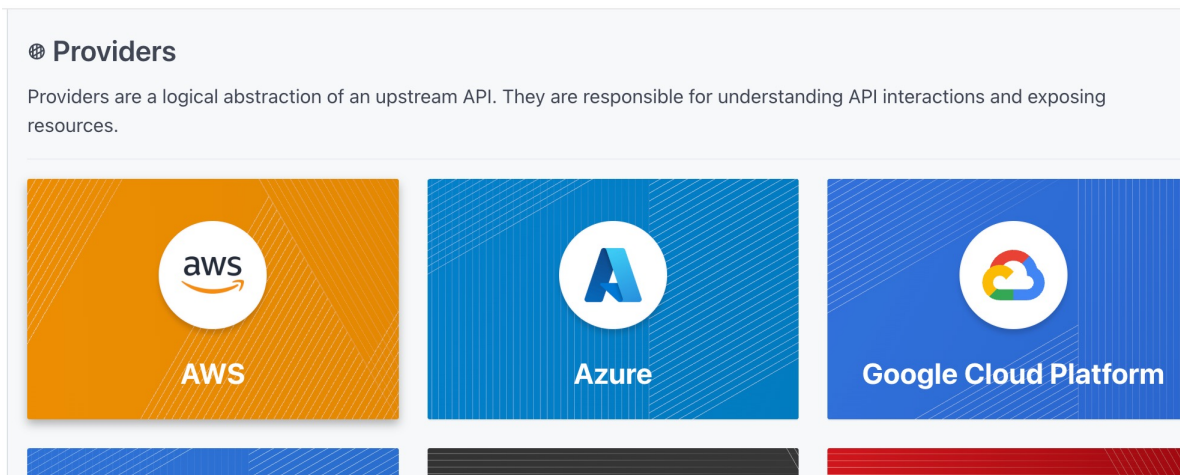
Architecture Terraform

Providers :

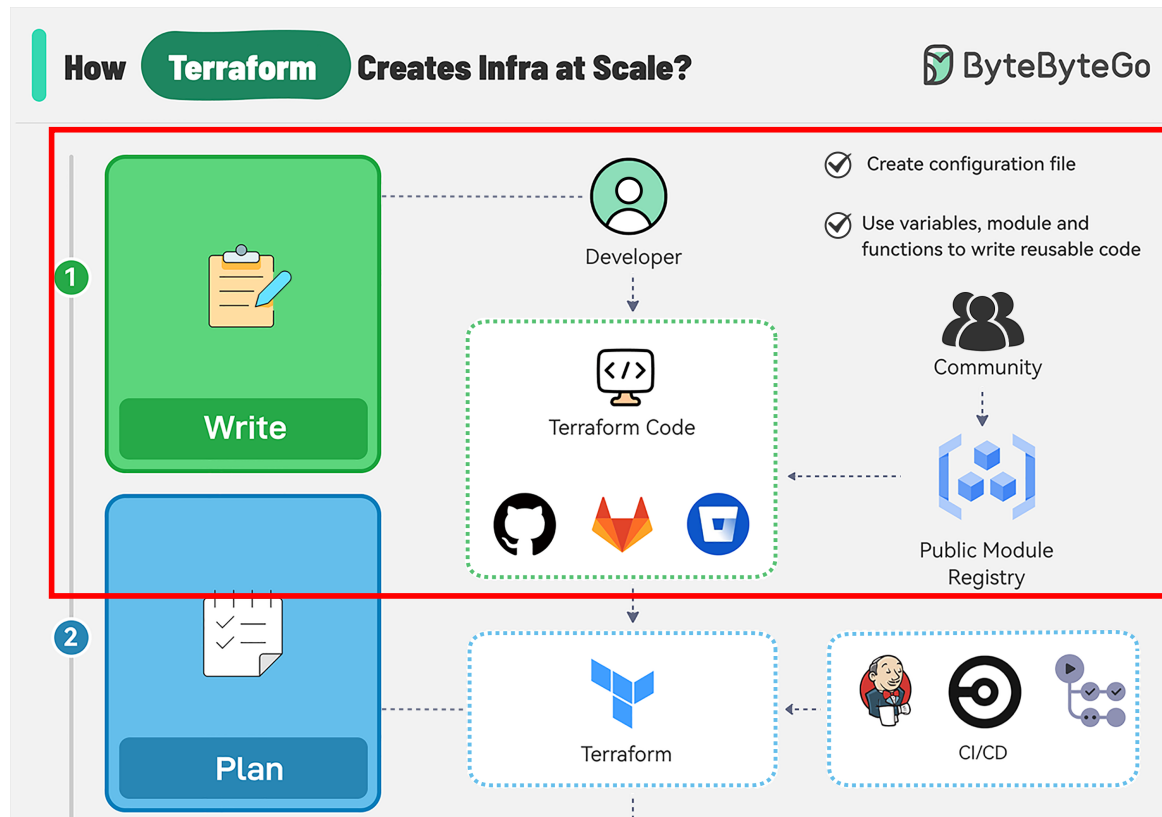
Les fournisseurs Terraform sont essentiellement des **plugins** que Terraform installe pour interagir avec des systèmes distants, tels qu'Azure, AWS, Google Cloud, Vmware, ...

Ces providers servent **d'interface** entre Terraform et l'infrastructure cible, permettant à Terraform de gérer les ressources souhaitées.

Chaque provider du **registre Terraform** dispose d'une **documentation** détaillant les ressources disponibles et leurs options de configuration.



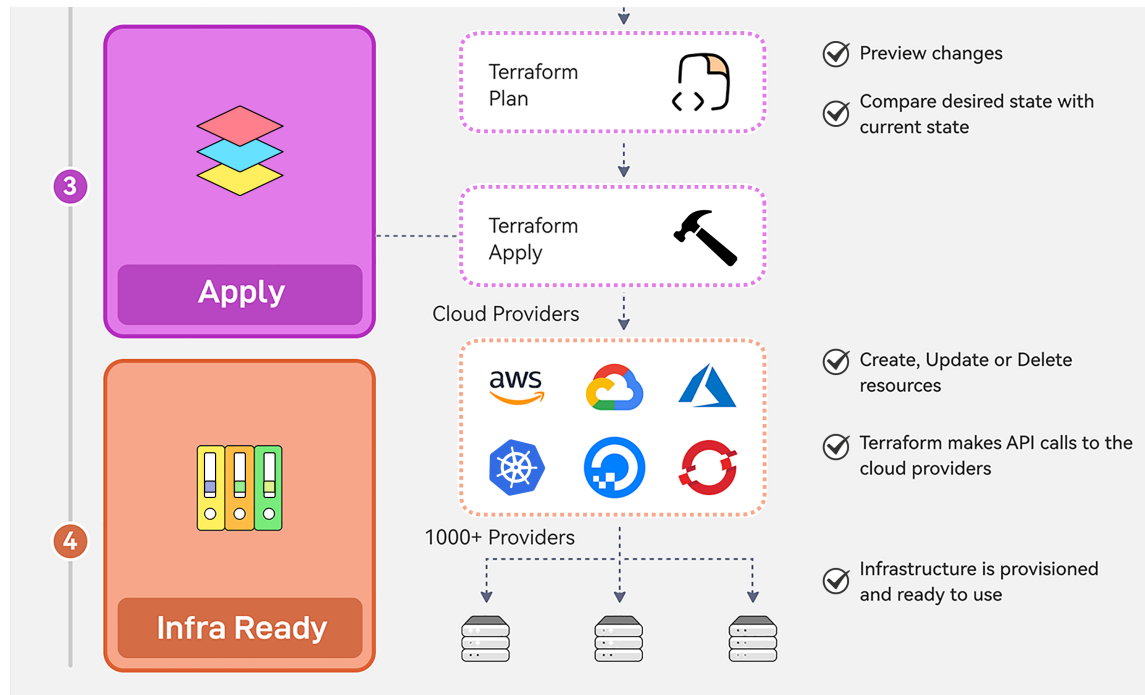
Workflow de déploiement



La commande `terraform plan` a pour but de **comparer** le code au **dernier état** d'exécution connu (le **state**) et de calculer le **plan d'exécution** qui permet d'atteindre l'état déclaré par le code.

`terraform plan`

Workflow de déploiement



La liste des opérations à effectuer suite à la comparaison entre le code et le state est appelée le **plan d'exécution (Terraform)**.

```
terraform apply
```

La commande `terraform apply` exécute un plan d'exécution.

Commandes utiles

```
> terraform -h
```

```
Usage: terraform [global options] <subcommand> [args]
```

The available commands for execution are listed below.

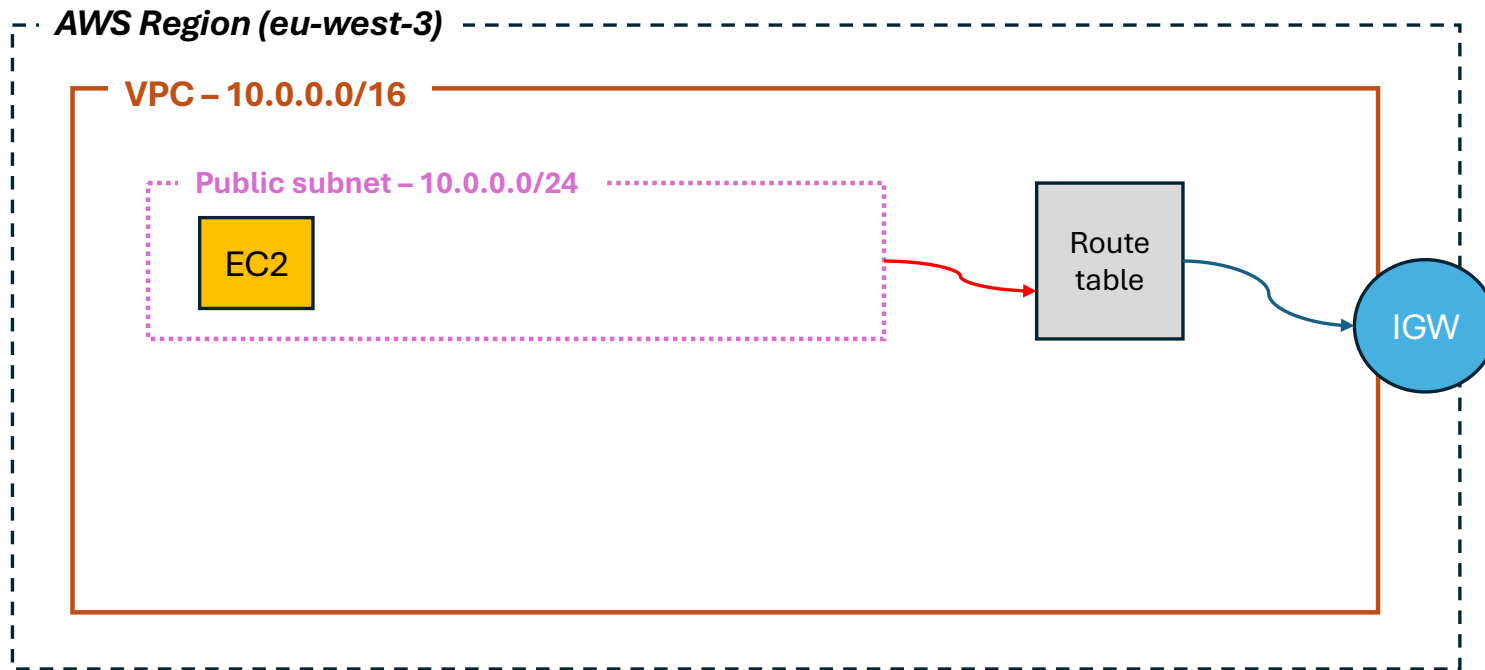
The primary workflow commands are given first, followed by less common or more advanced commands.

Main commands:

init	Prepare your working directory for other commands
validate	Check whether the configuration is valid
plan	Show changes required by the current configuration
apply	Create or update infrastructure
destroy	Destroy previously-created infrastructure

Fichier de configuration et syntaxe

Infrastructure à implémenter



Fichier de configuration et syntaxe

Ressources

Les ressources sont le bloc le plus couramment utilisé lors du codage dans HCL.

La définition d'un bloc comporte **trois parties** : le **type de bloc**, le **type de ressource** et le **nom de référence**.

Syntaxe en block

```
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {  
  # Block body  
  <ATTRIBUTE> = <EXPRESSION> # Argument  
}
```

Fichier de configuration et syntaxe

The screenshot shows the Terraform Registry page for the `aws_instance` resource. The header is blue with the Terraform logo and a search bar. Below the header, there are navigation links for Providers, hashicorp, aws, and Version 6.16.0. The main content area is divided into two columns. The left column, titled 'AWS DOCUMENTATION', contains a list of AWS resources with 'aws_instance' highlighted. The right column, titled 'Resource: aws_instance', contains a description of the resource, an 'Example Usage' section, and a 'Basic example using AMI lookup' section. The 'Basic example using AMI lookup' section shows a Terraform configuration snippet for creating an EC2 instance using an AMI lookup.

Providers / hashicorp / aws / Version 6.16.0 ▾ Latest Version

aws  Overview

AWS DOCUMENTATION

Q Filter

- aws_ec2_host
- aws_ec2_image_block_public_access
- aws_ec2_instance_connect_endpoint
- aws_ec2_instance_metadata_defaults
- aws_ec2_instance_state
- aws_ec2_serial_console_access
- aws_ec2_tag
- aws_eip
- aws_eip_association
- aws_eip_domain_name
- aws_instance**

Resource: aws_instance

Provides an EC2 instance resource. This allows instances to be created, updated, and deleted. Instances also support [provisioning](#).

Example Usage

Basic example using AMI lookup

Using a data source

```
data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}
```

Copy

Fichier de configuration et syntaxe

Dans cet exemple, le type de bloc est **resource**, le type de ressource est **aws_instance** et le nom de référence est **web_server**. Pour créer des **dépendances entre les ressources**, nous utilisons le **nom de référence** et le type pour accéder aux valeurs de sortie de la ressource :

```
resource « aws_instance » « web_server » {  
  
    name = « web-server »  
  
    ebs_volume {  
        size = 40  
    }  
}
```

Utilisé par Terraform

Utilisé par AWS

EBS = disque dur virtuel attaché à une machine EC2

Ressources = crée une ressource réelle (par exemple : EC2)

Fichier de configuration et syntaxe

Référence à un objet Terraform

<resource_type>.<name_label>.<attribut>

aws_instance.**web_server**.**name**

Fichier de configuration et syntaxe

Block data

Un block data permet de **récupérer des informations sur des ressources existantes** dans le cloud (ou d'autres providers), **sans les créer**.

```
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {  
  # Block body  
  <ATTRIBUTE> = <EXPRESSION> # Argument  
}
```

```
data "aws_ssm_parameter" "amzn2_linux" {  
  name = "aws/service/ami-amazon-linux-latest/amzn2-ami-hvn-x86_64-gp2"  
}
```

`aws_ssm_parameter` est une **data source AWS** fournie par le provider `aws`.

Ce block permet à Terraform de **lire une valeur stockée dans le SSM Parameter Store**, sans rien créer.

Fichier de configuration et syntaxe

Pour **réutiliser cette valeur** dans une ressource Terraform, par exemple pour lancer une instance EC2 avec la dernière image **Amazon Linux 2**.

```
resource "aws_instance" "web" {  
  ami = data.aws_ssm_parameter.amzn2_linux.value ← Récupère ID AMI  
  instance_type = "t3.micro"  
}
```

Il n'est pas nécessaire de mettre à jour le code quand AWS publie une nouvelle AMI.

AMI signifie **Amazon Machine Image**.

C'est une **image système préconfigurée** utilisée pour **lancer des instances EC2** (machines virtuelles AWS).

Fichier de configuration et syntaxe

Nom	Description	ID (exemple)
Amazon Linux 2	Linux optimisé pour EC2	ami- 0abcdef1234567890
Ubuntu 22.04	Ubuntu Server LTS	ami- 0123456789abcdef0

Fichier de configuration et syntaxe

Bloc terraform

Le bloc `terraform` permet de **configurer le comportement général** de Terraform :

- quelles versions utiliser,
- quels providers sont nécessaires,
- comment gérer le backend (stockage de l'état, par défaut en local)

```
terraform {  
  required_version = ">= 1.5.0"  
  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```

← Spécifie **quelle version minimale (ou exacte)** de Terraform doit être utilisée.

← Déclare les **providers (plugins)** nécessaires pour le projet

← où le trouver (le namespace officiel)

← quelle version du provider utiliser

Fichier de configuration et syntaxe

Bloc provider

Le **bloc provider** permet de **configurer la connexion entre Terraform et AWS**.

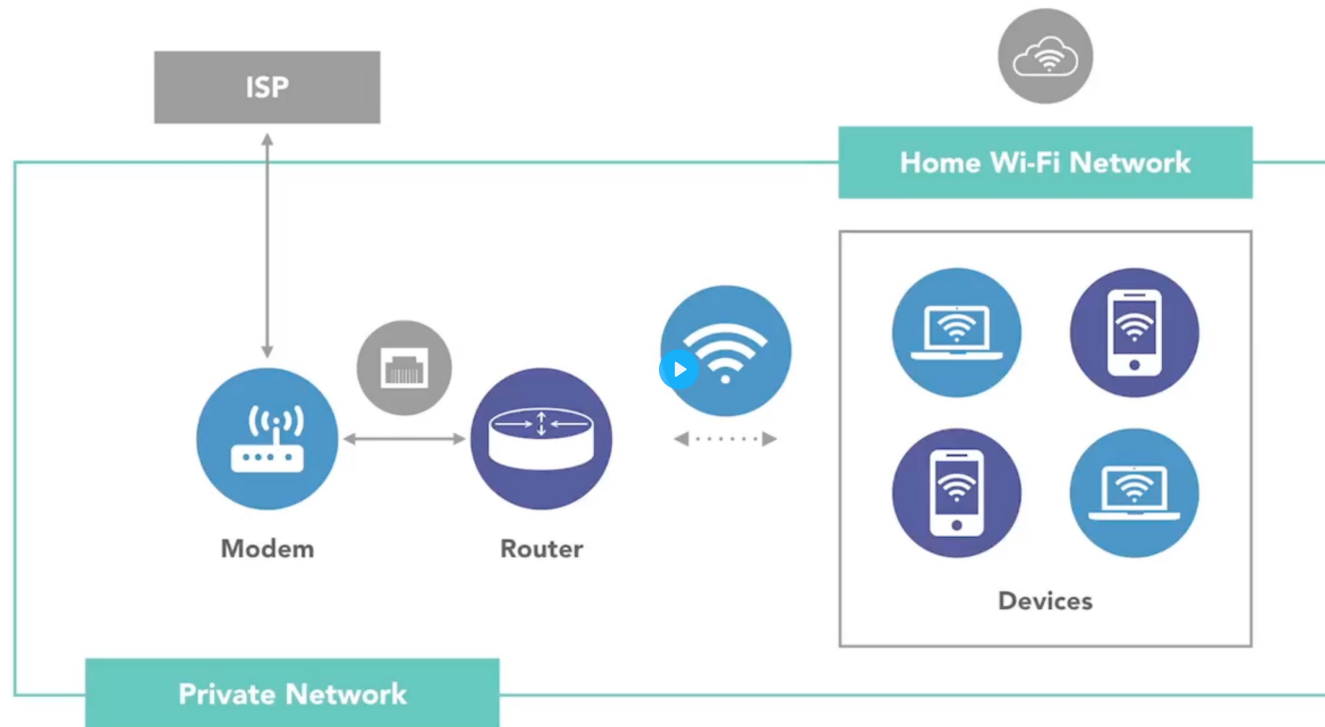
```
provider "aws" {  
  acces_key = " "  
  secret_key = " "  
  region = "us-east-1"  
}
```

Configuration de la connexion :

- Terraform va utiliser le provider **aws** (installé via le bloc `terraform.required_providers`).
- La région utilisée (us-east-1).
- Spécifier le compte AWS (`acces_key` et `secret_key`)

Fichier de configuration et syntaxe

Ressources réseaux – Réseau privé (domestique)



The diagram illustrates a Private Network architecture within a Virtual Private Cloud (VPC). The VPC is represented by a large teal rectangle. Inside the VPC, there is a Private Network (teal rectangle at the bottom) and a large white rectangle labeled "AWS Resources". The Private Network contains an Internet Gateway (blue circle with a router icon) and a Route Table (blue circle with a router icon). The Internet Gateway is connected to the AWS cloud (grey rectangle at the top) via a double-headed arrow. The Route Table is connected to the Internet Gateway via a double-headed arrow. A play button icon is positioned between the Route Table and the AWS Resources rectangle, indicating a connection or action. A dashed double-headed arrow connects the Route Table to the AWS Resources rectangle. A small grey circle with a Wi-Fi icon is located above the AWS Resources rectangle.

Fichier de configuration et syntaxe

Ressources réseaux - VPC (Virtual Private Cloud)

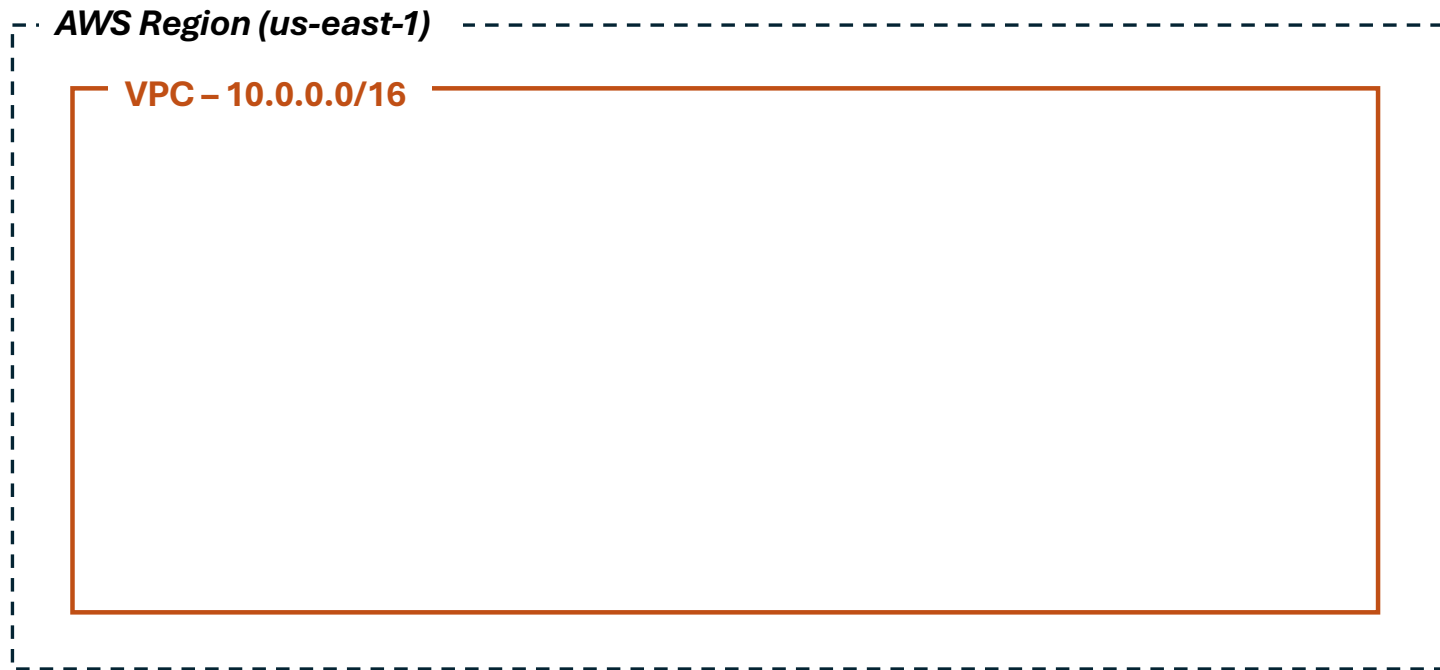
```
resource "aws_vpc" "app" {  
  cidr_block      = "10.0.0.0/16"  
  enable_dns_hostnames = true  
}
```

Rôle de VPC : est un service AWS qui permet de créer un réseau virtuel isolé dans le cloud AWS où vous pouvez lancer vos ressources AWS (EC2, RDS, etc.).

- **CIDR block :** `10.0.0.0/16` signifie que ton VPC peut avoir des adresses IP de `10.0.0.0` à `10.0.255.255`.
- **enable_dns_hostnames = true :** Permet d'attribuer des noms DNS aux instances EC2 dans ce VPC.

Fichier de configuration et syntaxe

Ressources réseaux - VPC (Virtual Private Cloud)



Fichier de configuration et syntaxe

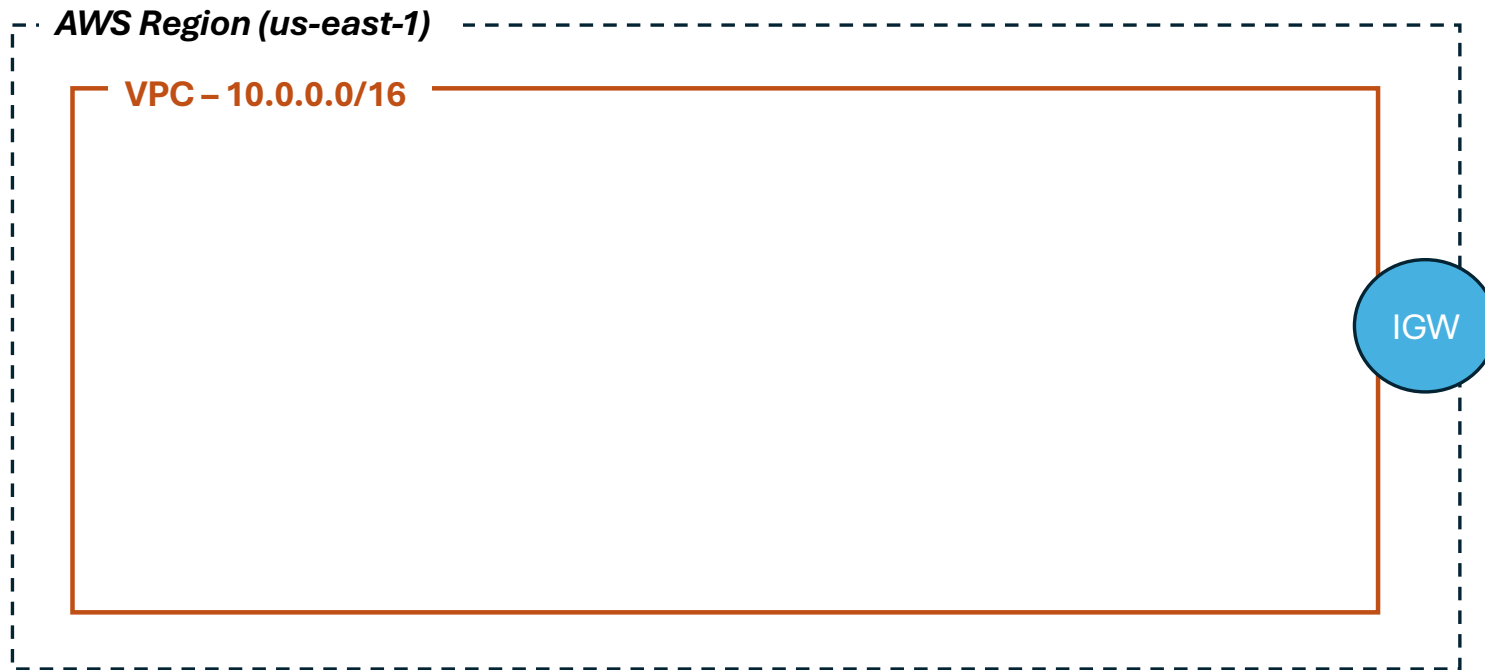
Ressources réseaux - Internet Gateway (passerelles Internet)

```
resource "aws_internet_gateway" "app" {  
  vpc_id = aws_vpc.app.id  
}
```

- **Rôle** : Permet aux ressources de votre VPC d'accéder à Internet et aux clients externes d'accéder à vos ressources publiques.
- Internet Gateway est attaché au VPC créé précédemment.

Fichier de configuration et syntaxe

Ressources réseaux - Internet Gateway (passerelles Internet)



Fichier de configuration et syntaxe

Ressources réseaux - Subnet (sous-réseau)

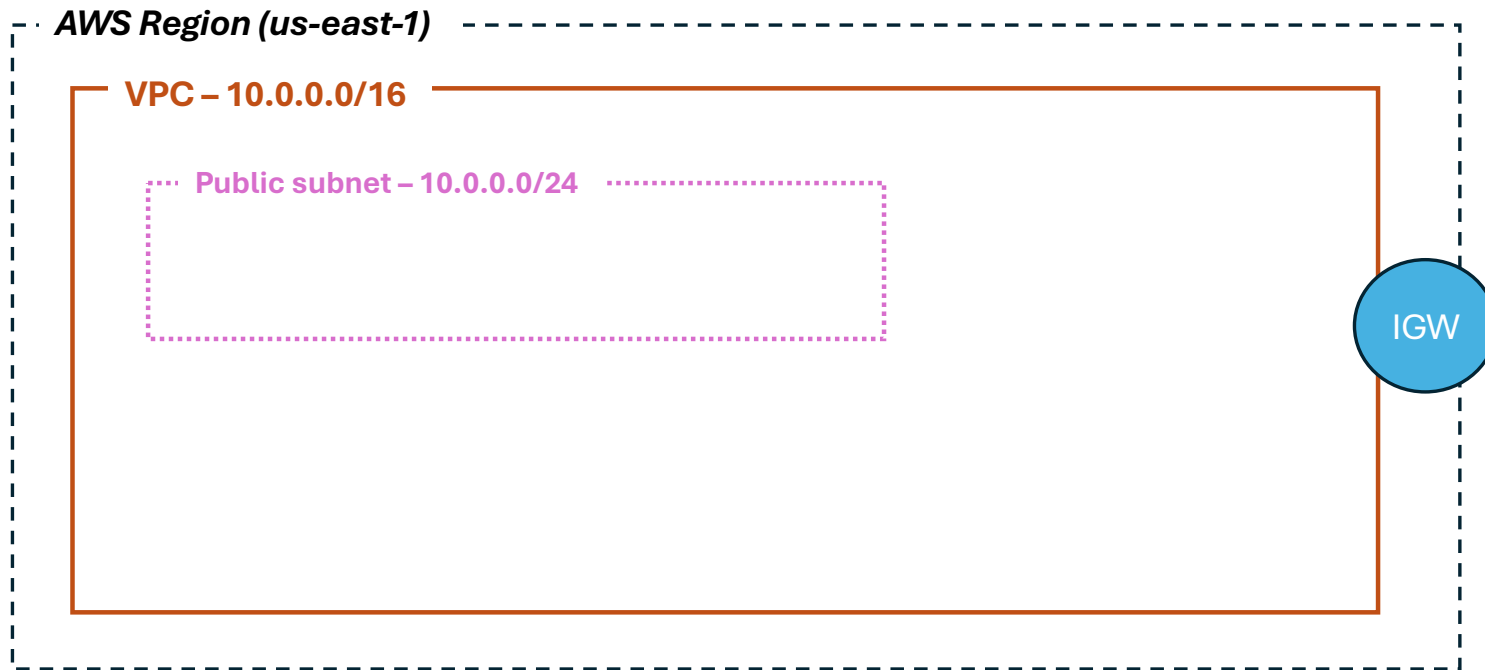
```
resource "aws_subnet" "public_subnet1" {  
  cidr_block      = "10.0.0.0/24"  
  vpc_id          = aws_vpc.app.id  
  map_public_ip_on_launch = true  
}
```

Rôle : Divisions logiques d'un réseau VPC dans des segments plus petits à l'intérieur d'une zone de disponibilité AWS.

- **CIDR block :** 10.0.0.0/24 → 256 adresses IP disponibles pour ce sous-réseau.
- **map_public_ip_on_launch = true :** Les instances lancées dans ce subnet recevront automatiquement une IP publique, ce qui est nécessaire pour accéder à Internet.

Fichier de configuration et syntaxe

Ressources réseaux - Subnet (sous-réseau)



Fichier de configuration et syntaxe

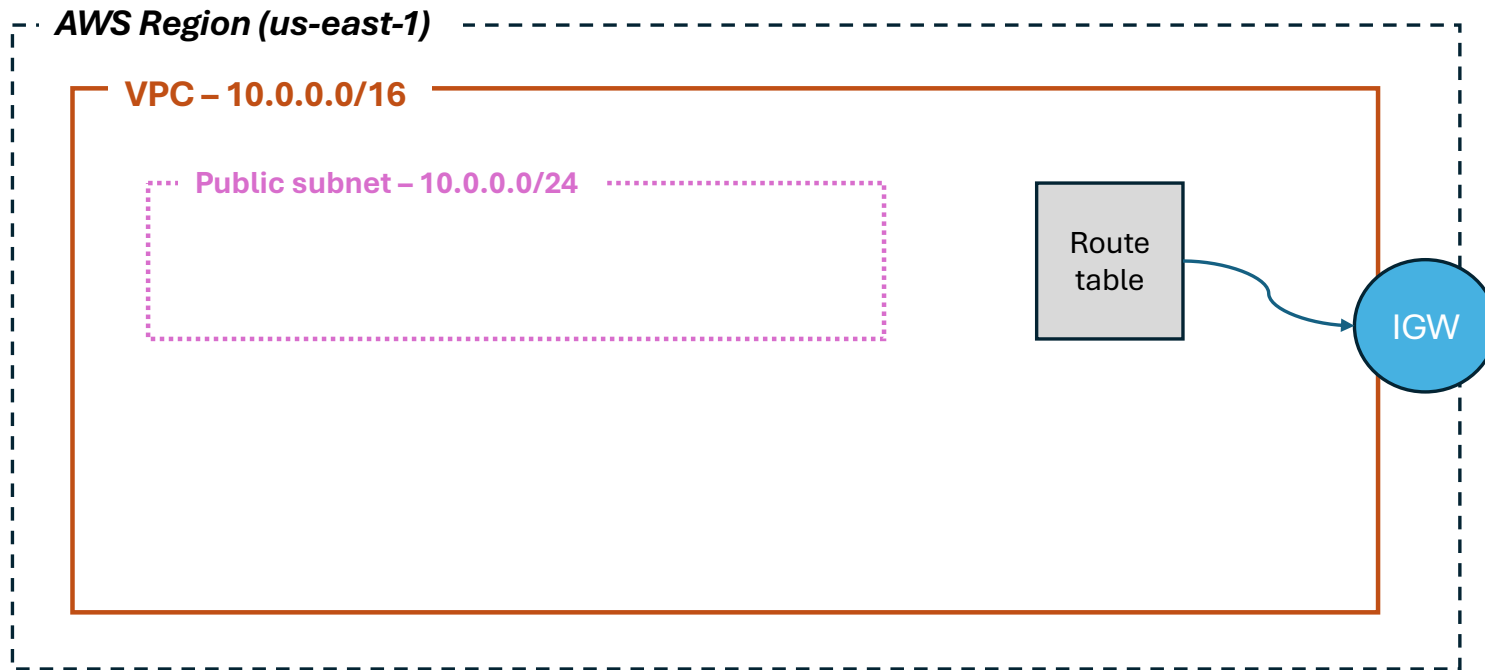
ROUTAGE - Route table (table de routage)

```
resource "aws_route_table" "app" {  
  vpc_id = aws_vpc.app.id  
  
  route {  
    cidr_block = "0.0.0.0/0"  
    gateway_id = aws_internet_gateway.app.id  
  }  
}
```

- **Rôle** : Définit comment le trafic réseau sortant est dirigé.
- **route** :
 - 0.0.0.0/0 → toutes les destinations.
 - gateway_id = aws_internet_gateway.app.id → tout le trafic sortant va vers l'Internet Gateway (accès Internet).

Fichier de configuration et syntaxe

Routage - Route table (table de routage)



Fichier de configuration et syntaxe

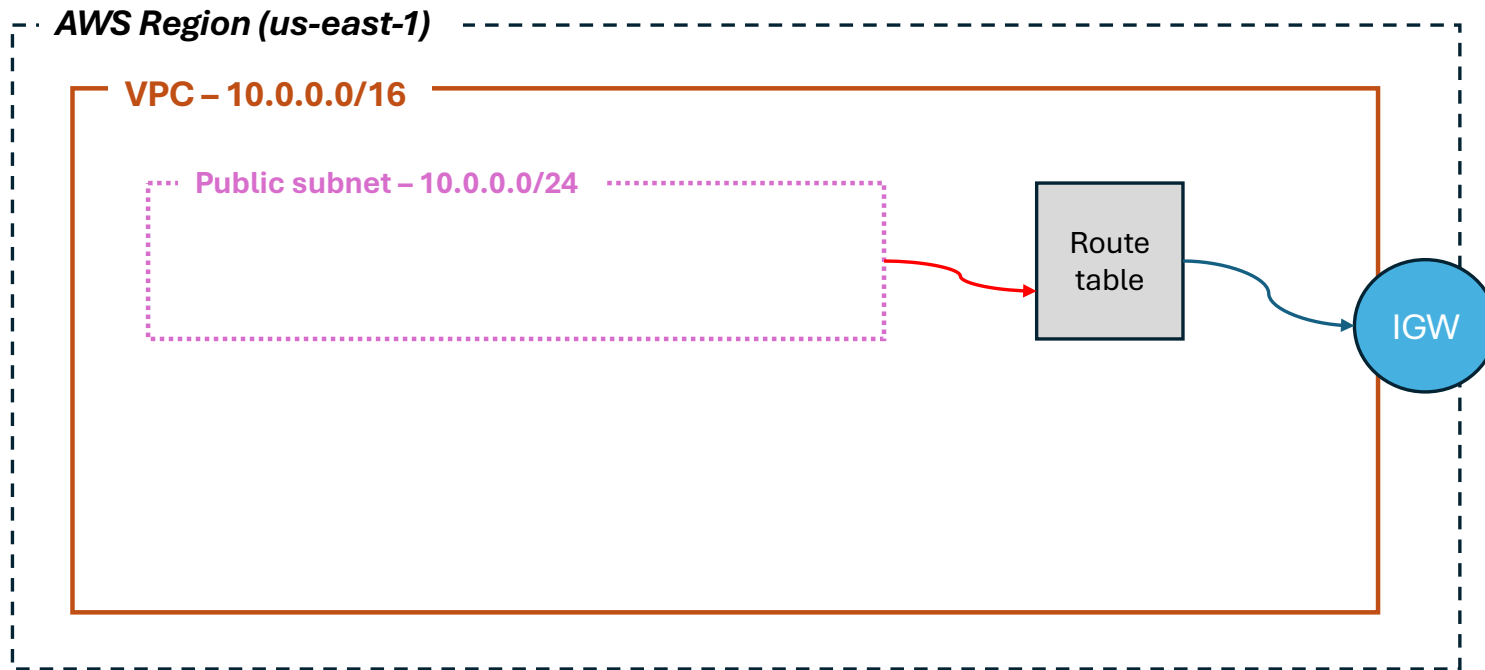
Routage - Association de la Route Table au Subnet

```
resource "aws_route_table_association" "app_subnet1" {  
  subnet_id    = aws_subnet.public_subnet1.id  
  route_table_id = aws_route_table.app.id  
}
```

- **Rôle** : Lie la table de routage au subnet.
- Cela permet à toutes les instances dans `public_subnet1` d'utiliser la route définie pour accéder à Internet.

Fichier de configuration et syntaxe

Routage - Association de la Route Table au Subnet



Fichier de configuration et syntaxe

Sécurité – Security group (groupe de sécurité)

```
resource "aws_security_group" "nginx_sg" {  
  name = "nginx_sg"  
  vpc_id = aws_vpc.app.id
```

Un **security group** est un firewall au niveau des ressources (stateful).

Il détermine :

- quel trafic **entrant (ingress)** est autorisé
- quel trafic **sortant (egress)** est autorisé

Fichier de configuration et syntaxe

Sécurité – Security group (groupe de sécurité)

Règles Ingress (trafic entrant)

```
ingress {  
  from_port = 80  
  to_port   = 80  
  protocol  = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}
```

Cette règle fait :

- Autorise le trafic HTTP (port 80)
- Depuis n'importe où dans le monde (0 . 0 . 0 . 0 / 0)
- En TCP

Fichier de configuration et syntaxe

Sécurité – Security group (groupe de sécurité)

Règles Egress (trafic sortant)

```
egress {  
  from_port = 0  
  to_port   = 0  
  protocol  = "-1"  
  cidr_blocks = ["0.0.0.0/0"]  
}
```

Cette règle fait :

- Autorise **tout** le trafic sortant
- Vers **toutes les IP**
- Pour **n'importe quel protocole** (-1 = all protocols)

Cela permet à ton serveur de :

- contacter Internet
- télécharger des packages
- accéder à une base donnée externe
- effectuer des requêtes sortantes en général

Fichier de configuration et syntaxe

Instance – Ressource AWS

```
resource "aws_instance" "nginx1" {  
  ami          = data.aws_ssm_parameter.amzn2_linux.value  
  instance_type = "t3.micro"  
  subnet_id    = aws_subnet.public_subnet1.id  
  vpc_security_group_ids = [aws_security_group.nginx_sg.id]  
  user_data_replace_on_change = true  
}
```

instance_type = "t3.micro" :

- Petite instance généraliste
- 2 vCPU / 1 Go RAM
- Souvent utilisée dans les environnements de test ou Free Tier

Fichier de configuration et syntaxe

Instance – Ressource AWS

```
resource "aws_instance" "nginx1" {  
  ami          = data.aws_ssm_parameter.amzn2_linux.value  
  instance_type = "t3.micro"  
  subnet_id    = aws_subnet.public_subnet1.id  
  vpc_security_group_ids = [aws_security_group.nginx_sg.id]  
  user_data_replace_on_change = true  
}
```

subnet_id = aws_subnet.public_subnet1.id :

- Cette instance est lancée dans **un subnet public**.
- Puisqu'il est public et lié à un Internet Gateway, l'instance peut :
 - recevoir une IP publique
 - être accessible depuis l'extérieur

Fichier de configuration et syntaxe

Instance – Ressource AWS

```
resource "aws_instance" "nginx1" {  
  ami          = data.aws_ssm_parameter.amzn2_linux.value  
  instance_type = "t3.micro"  
  subnet_id    = aws_subnet.public_subnet1.id  
  vpc_security_group_ids = [aws_security_group.nginx_sg.id]  
  user_data_replace_on_change = true  
}
```

vpc_security_group_ids = [aws_security_group.nginx_sg.id] :

- L'instance utilise le security group défini plus haut

user_data_replace_on_change = true :

- Si on modifie le script user_data, Terraform remplacera automatiquement l'instance EC2.

Fichier de configuration et syntaxe

Instance – Ressource AWS

```
user_data = <<EOF
#!/bin/bash
sudo amazon-linux-extras install -y nginx1
sudo service nginx start
```

- Terraform injecte ce script dans l'instance. C'est exécuté **au premier démarrage**.
- `amazon-linux-extras` est un **outil de gestion de paquets** spécifique à **Amazon Linux 2**

Fichier de configuration et syntaxe

Instance – Ressource AWS

```
sudo cat > /usr/share/nginx/html/index.html << 'WEBSITE'
<html>
<head>
  <title>HeH - DST</title>
</head>
<body style="background-color:#1F778D">
  <p style="text-align: center;">
    <span style="color:#FFFFFF;">
      <span style="font-size:100px;">Welcome to the website!
Have a &#127790;</span>
    </span>
  </p>
</body>
</html>
WEBSITE
EOF
}
```

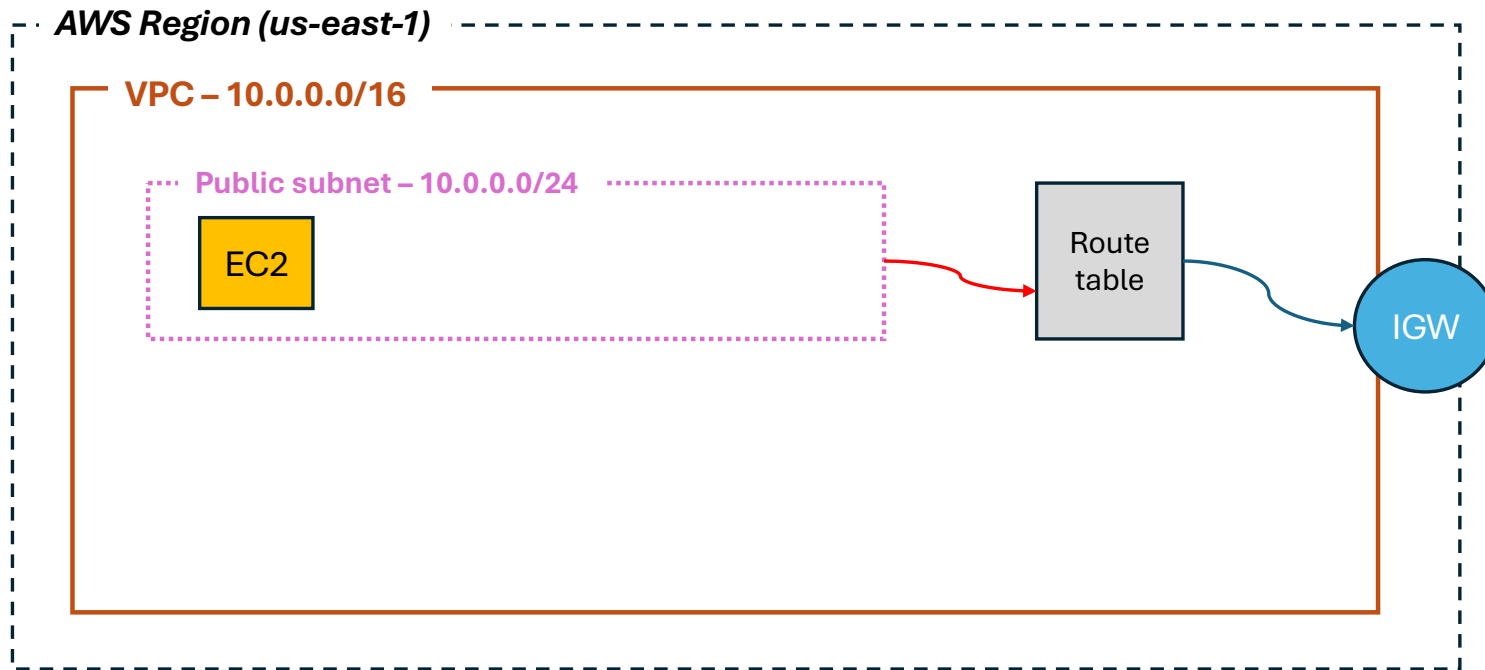
Le << 'WEBSITE' est un **heredoc**, qui permet d'insérer un fichier complet directement dans Terraform.

Il y a même **deux heredocs imbriqués** :

- 1 . <<EOF → Début du script EC2 user data
- 2 . << 'WEBSITE' → Insertion de la page HTML complète

Fichier de configuration et syntaxe

Instance – Ressource AWS



Déploiement de l'infra avec Terraform

Initialisation de la configuration

```
terraform init
```

Lorsque cette commande est exécutée, Terraform réalise principalement **trois actions essentielles** :

1. Configure le backend de stockage (local ou distant)
=> Si le backend n'est pas spécifié, Terraform créera le fichier `terraform.tfstate` dans le dossier courant.
2. Télécharge et installe tous les **providers** déclarés dans le fichier `main.tf`
3. Télécharge et prépare les modules



Déploiement de l'infra avec Terraform

Initialisation de la configuration

Exemples/Terraform/base_web_app via  default on  (us-east-1)

```
> terraform init
```

Initializing the backend...

Initializing provider plugins...

- Finding hashicorp/aws versions matching "~> 5.0"...

- Installing hashicorp/aws v5.100.0...

- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

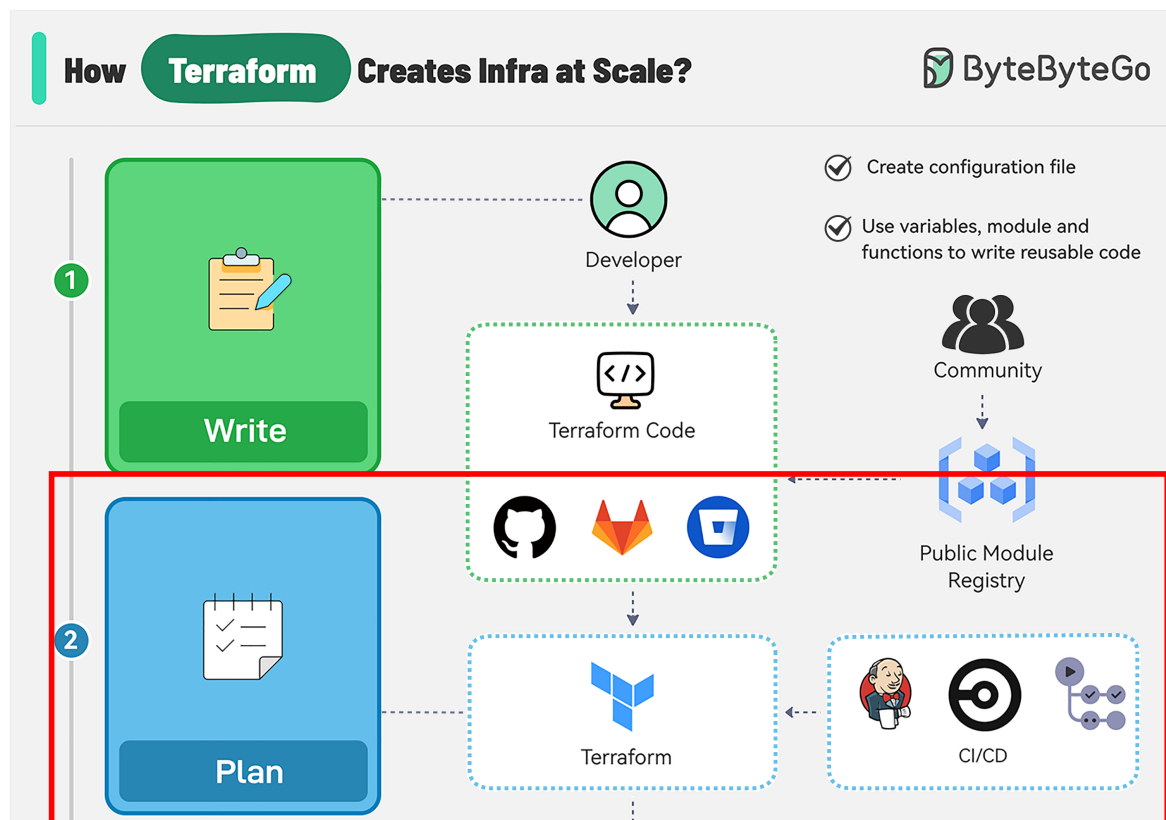
Terraform has created a lock file **.terraform.lock.hcl** to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

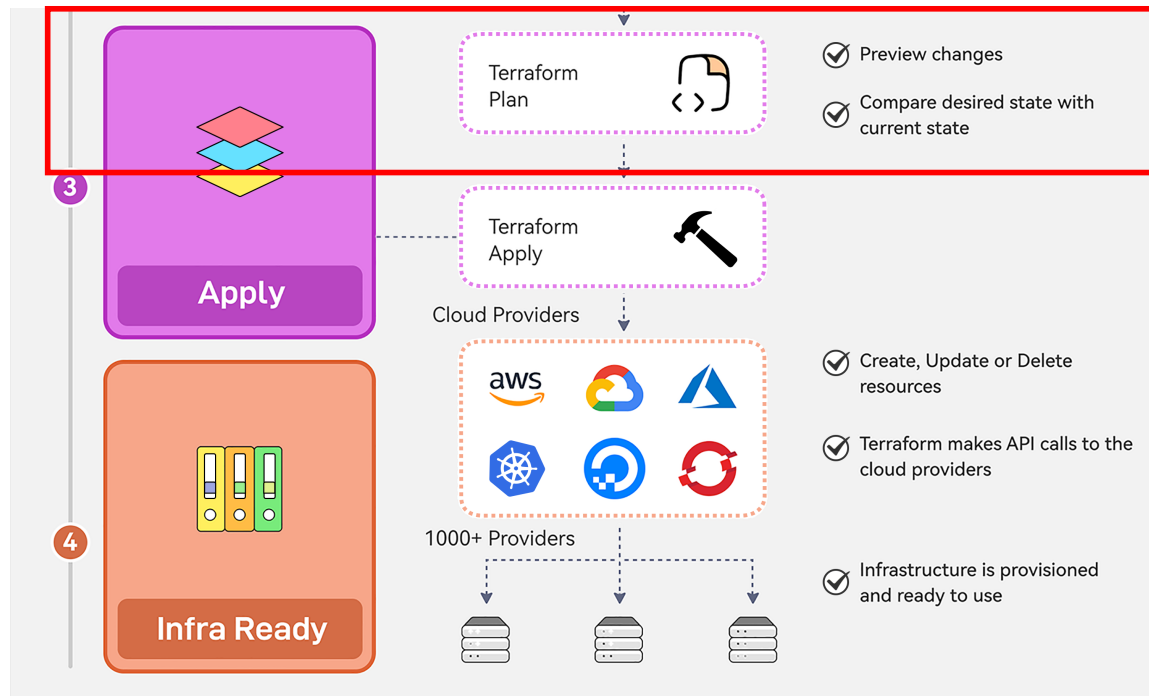
You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

Déploiement de l'infra avec Terraform



Déploiement de l'infra avec Terraform



La liste des opérations à effectuer suite à la comparaison entre le code et le state est appelée le **plan d'exécution (Terraform)**.

Déploiement de l'infra avec Terraform

Planification du déploiement

```
terraform plan
```

La commande a pour but de **comparer** le code au **dernier état** d'exécution connu (le **state**) et de calculer le **plan d'exécution** qui permet d'atteindre l'état déclaré par le code.

1. Lecture et analyse de l'état actuel (state)

- Terraform lit l'**état actuel** de l'infrastructure (`terraform.tfstate`)

2. Analyse de la configuration et création du plan

- Terraform compare :
 - la **configuration déclarée** dans les fichiers .tf
 - l'**état réel** de l'infrastructure
- Cela génère un plan d'exécution listant les actions suivantes :
 - **Créer** des ressources
 - **Modifier** des ressources
 - **Supprimer** des ressources

Attention : Pour exécuter cette commande, vous devez être identifié sur AWS avec votre compte.

Déploiement de l'infra avec Terraform

Planification du déploiement

3. Affichage du plan détaillé (sans rien changer)

Terraform affiche ensuite le plan sous forme de résumé lisible :

- + pour **création**
- ~ pour **modification**
- - pour **suppression**

Pour sauvegarder le plan dans un fichier, ajouter l'option -out :

```
terraform plan -out nomFichier.tfplan
```

Pour afficher le plan :

```
terraform show ./nomFichier.tfplan
```

Déploiement de l'infra avec Terraform

Planification du déploiement

Authentification avec AWS CLI

```
> aws configure
AWS Access Key ID [*****WATQ]: ASIA3PRZGSENGFJU7QM7
AWS Secret Access Key [*****bHew]: 2/EN7/Ph0j3stCgHTd97iy+INa2quXTFJ/93p04s
AWS Session Token [*****1g==]: IQoJb3JpZ2luX2VjEBMaCXVzLXdlc3QtMiJIMEYCIQC24hHmLpEWAd+
y/CgIhALCQVjeXRAoAytT4GzsIPPacS0dII+klavGt+Kdw4YB5KrMCCNz////////wEQARoMNzg5MzIwMjA4NjY2IgwRn4t
+bqUZq6qFoaltl5anYhqvkqKLRUFLug0sDkGfWfTxaMoyXBNvH/kiB0nYR0Q0YSXLZBPVVo7J4eziXatL9DEyizVbS4ZQyDc
73h+UUW0u4u0V3xDTS0dWHY3uZt0yqUZHbLKWIJZP39UB9GCLQN4kq5cXpQq7rAKB0P4jphM5M0HnTsPCRPa96HRNt/SWpIF3
/jBx43ReVIQd9umi0ATlojhE5LKu1B+LQD3TM8IpyRBfJmjDPsOG0m+4nKPt5gNmJfLghCwRA9cg15CtnqHE/bemyYeDC1v/k
G0zJk+0Yz4GK1/eERApjru3fmt2xEpF8mQFd56iZiY8LMVopYiRsZAlPro+TzogJwc/da39WAdljfLUCd22XSUHoHN6xpd00l
kPXMFDlfX9rtSQafux6V0DJnRdCkCQoMeKe2nC1rziWpdTrbAAI7YNzMijLzTiRQ==
Default region name [us-east-1]:
Default output format [json]:
```

Exemples/Terraform/base_web_app via  default on  (us-east-1) took 1m44s

```
> █
```

Déploiement de l'infra avec Terraform

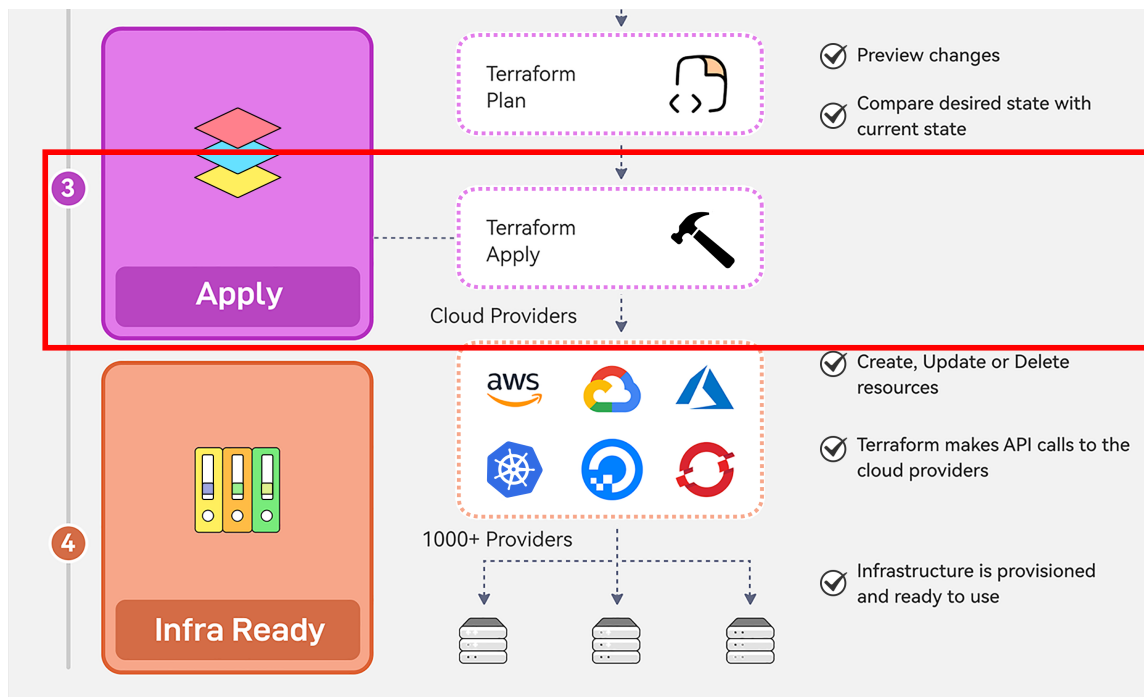
Planification du déploiement

Authentification avec les variables d'environnement

```
export AWS_ACCESS_KEY_ID="..."  
export AWS_SECRET_ACCESS_KEY="..."  
export AWS_SESSION_TOKEN="..."
```

Déploiement de l'infra avec Terraform

Déploiement de l'infra



La commande `terraform apply` exécute le plan d'exécution.

Cette commande est celle qui **exécute réellement les changements** sur votre infrastructure.

Déploiement de l'infra avec Terraform

Déploiement de l'infra

Après l'exécution de la commande :

1. Terraform exécute les actions selon la planification
2. Mise à jour du fichier terraform.tfstate

Cette commande permet de fournir à Terraform le plan généré par l'étape précédente :

```
terraform apply ./ nomFichier.tfplan
```

Si vous n'avez pas passé un plan pré-enregistré, Terraform relance une planification de déploiement (plan)

Déploiement de l'infra avec Terraform

Déploiement de l'infra

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

```
aws_instance.nginx1: Creating...  
aws_instance.nginx1: Still creating... [00m10s elapsed]  
aws_instance.nginx1: Still creating... [00m20s elapsed]  
aws_instance.nginx1: Still creating... [00m30s elapsed]  
aws_instance.nginx1: Still creating... [00m40s elapsed]  
aws_instance.nginx1: Creation complete after 44s [id=i-09ea7d1e6b1b77323]
```

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Demande de confirmation

Exécution des actions

Déploiement de l'infra avec Terraform

Déploiement de l'infra

EC2 > Instances

EC2 <

Tableau de bord
Vue globale EC2 [L](#)
Événements
▼ Instances

Instances (2) Informations

Date de la dernière mise à jour : Il y a 14 minutes

Se connecter État de l'instance Actions Lancer des instances ▼

Rechercher Instance par attribut ou identification (case-sensitive) Tous les é... ▼

☐	Name 🔗	ID d'instance	État de l'insta... ▼	Type d'insta... ▼	Contrôle des statu	Statut d'alarme	Zone de dispon... ▼	DNS IPv4 public ▼	Adresse IP
☐	Bastion Host	i-0aa11e5d9ec8c7d49	✓ En cours d'... 🔗	t2.micro	✓ 2/2 vérifications r	Afficher les alarme	us-east-1a	ec2-54-147-179-239.co...	54.147.17!
☐		i-09ea7d1e6b1b77323	✓ En cours d'... 🔗	t2.micro	✓ 2/2 vérifications r	Afficher les alarme	us-east-1e	ec2-54-157-165-233.co...	54.157.16!

Déploiement de l'infra avec Terraform

Déploiement de l'infra

EC2 > Instances > i-09ea7d1e6b1b77323

EC2

- Tableau de bord
- Vue globale EC2
- Événements
- ▼ **Instances**
 - Instances
 - Types d'instances
 - Modèles de lancement
 - Demandes Spot
 - Savings Plans
 - Instances réservées
 - Hôtes dédiés
 - Réservations de capacité
 - Capacity Manager
- ▼ **Images**
 - AMI

Résumé de l'instance pour i-09ea7d1e6b1b77323 Informations

Mis à jour il y a 1 minute

ID d'instance
i-09ea7d1e6b1b77323

Adresse IPv6
-

Type de nom d'hôte
Nom de l'adresse IP: ip-10-0-0-203.ec2.internal

Réponse à un nom DNS de ressource privée
-

Adresse IP attribuée automatiquement
54.157.165.233 [IP publique]

Rôle IAM
-

Adresse IPv4 publique
54.157.165.233 | [adresse ouverte](#)

État de l'instance
En cours d'exécution

Nom DNS de l'IP privé (IPv4 uniquement)
ip-10-0-0-203.ec2.internal

Type d'instance
t2.micro

ID de VPC
vpc-0c7257ee3ca4f7df3

ID de sous-réseau
subnet-0f43a43b30d689c8a

Se connecter **État de l'instance** **Actions**

Adresses IPv4 privées
10.0.0.203

DNS public
ec2-54-157-165-233.compute-1.amazonaws.com | [adresse ouverte](#)

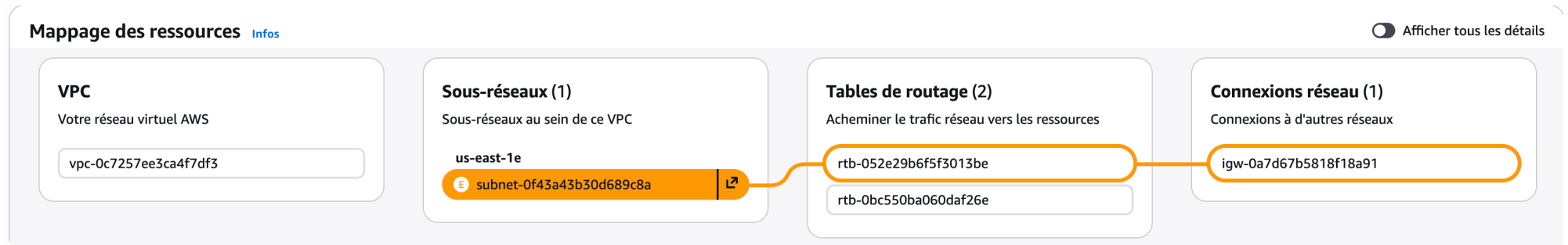
Adresses IP élastiques
-

Recherche d'AWS Compute Optimizer
Inscrivez-vous à AWS Compute Optimizer pour obtenir des recommandations.
[En savoir plus](#)

Nom du groupe Auto Scaling
-

Déploiement de l'infra avec Terraform

Déploiement de l'infra



Déploiement de l'infra avec Terraform

Suppression de l'infra

La commande **terraform destroy** sert à **supprimer toute l'infrastructure** gérée par Terraform. Elle fait l'inverse de `terraform apply` : au lieu de créer/modifier, elle **détruit** les ressources.

```
> terraform destroy
data.aws_ssm_parameter.amzn2_linux: Reading...
aws_vpc.app: Refreshing state... [id=vpc-0c7257ee3ca4f7df3]
data.aws_ssm_parameter.amzn2_linux: Read complete after 0s [id=/aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2]
]
aws_internet_gateway.app: Refreshing state... [id=igw-0a7d67b5818f18a91]
aws_subnet.public_subnet1: Refreshing state... [id=subnet-0f43a43b30d689c8a]
aws_security_group.nginx_sg: Refreshing state... [id=sg-0178df9859c86766a]
aws_route_table.app: Refreshing state... [id=rtb-052e29b6f5f3013be]
aws_route_table_association.app_subnet1: Refreshing state... [id=rtbassoc-02fb6f3e2342dfd40]
aws_instance.nginx1: Refreshing state... [id=i-09ea7d1e6b1b77323]
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- destroy

Terraform will perform the following actions:

```
# aws_instance.nginx1 will be destroyed
- resource "aws_instance" "nginx1" {
  - ami              = "ami-06124b567f8becfbd" -> null
  - arn              = "arn:aws:ec2:us-east-1:789320208666:instance/i-09ea7d1e6b1b77323" -> null
  - associate_public_ip_address = true -> null
  - availability_zone = "us-east-1e" -> null
  - cpu_core_count    = 1 -> null
  - cpu_threads_per_core = 1 -> null
```

Génération d'un
plan de destruction

Déploiement de l'infra avec Terraform

Suppression de l'infra

Plan: 0 to add, 0 to change, 7 to destroy.

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

Demande de confirmation

```
aws_route_table_association.app_subnet1: Destroying... [id=rtbassoc-02fb6f3e2342dfd40]
aws_instance.nginx1: Destroying... [id=i-09ea7d1e6b1b77323]
aws_route_table_association.app_subnet1: Destruction complete after 1s
aws_route_table.app: Destroying... [id=rtb-052e29b6f5f3013be]
aws_route_table.app: Destruction complete after 1s
aws_internet_gateway.app: Destroying... [id=igw-0a7d67b5818f18a91]
aws_instance.nginx1: Still destroying... [id=i-09ea7d1e6b1b77323, 00m10s elapsed]
aws_internet_gateway.app: Still destroying... [id=igw-0a7d67b5818f18a91, 00m10s elapsed]
aws_instance.nginx1: Still destroying... [id=i-09ea7d1e6b1b77323, 00m20s elapsed]
aws_internet_gateway.app: Still destroying... [id=igw-0a7d67b5818f18a91, 00m20s elapsed]
aws_instance.nginx1: Still destroying... [id=i-09ea7d1e6b1b77323, 00m30s elapsed]
aws_internet_gateway.app: Destruction complete after 29s
aws_instance.nginx1: Still destroying... [id=i-09ea7d1e6b1b77323, 00m40s elapsed]
aws_instance.nginx1: Destruction complete after 42s
aws_subnet.public_subnet1: Destroying... [id=subnet-0f43a43b30d689c8a]
aws_security_group.nginx_sg: Destroying... [id=sg-0178df9859c86766a]
aws_subnet.public_subnet1: Destruction complete after 0s
aws_security_group.nginx_sg: Destruction complete after 1s
aws_vpc.app: Destroying... [id=vpc-0c7257ee3ca4f7df3]
aws_vpc.app: Destruction complete after 1s
```

Exécution de la destruction

Destroy complete! Resources: 7 destroyed.

Utilisation des Inputs et Outputs

Améliorations du code Terraform

- Beaucoup de valeurs codées en dur => utiliser des variables autant que possible.
- Il serait également utile de connaître le nom d'hôte DNS public de l'instance EC2 sans avoir à passer par la console AWS.
- Certaines informations VPC pour une utilisation ultérieure.

Utilisation des Inputs et Outputs

Variables d'entrée (Inputs variables) - Définition

Dans Terraform, les **variables d'entrée (Input Variables)** permettent de rendre votre configuration **flexible, réutilisable et paramétrable**.

Elles jouent un rôle essentiel pour éviter de “hardcoder” des valeurs dans vos fichiers `.tf`.

```
variable "name_label" {  
  type = value  
  description = "string"  
  default = value  
}
```

```
variable "instance_type" {  
  description = "Type de l'instance EC2"  
  type      = string  
  default   = "t2.micro"  
}
```

type : string, number, bool, list, map...

description : explique la variable (bonne pratique)

default (optionnel) : valeur par défaut → si absent, Terraform exigera une valeur

Utilisation des Inputs et Outputs

Variables d'entrée (Inputs variables) - Référence

```
var.instance_type
```

Pour récupérer la valeur de la variable « instance_type »

Type de données



Primitive

String, number, bool



Structural

Tuple, object



Collection

List, set, map

Utilisation des Inputs et Outputs

Variables d'entrée (Inputs variables) - Exemples

List

```
[1, 2, 3, 4]
```

```
["us-east-1", "us-east-2", "us-west-1", "us-west-2"]
```

```
[1, "us-east-2", true] # INVALID LIST!
```

Map

```
{  
  small = "t2.micro"  
  medium = "t2.small"  
  large = "t2.large"  
}
```

Utilisation des Inputs et Outputs

Variables d'entrée (Inputs variables) - Exemples

```
variable "aws_regions" {  
  type = list(string)  
  description = "Regions to use for AWS resources »  
  default = ["us-east-1", "us-east-2", "us-west-1", "us-west-2"]  
}
```

Récupérer une valeur d'une variable de type « list » : `var.aws_regions[0]`

Utilisation des Inputs et Outputs

Variables d'entrée (Inputs variables) - Exemples

```
variable "aws_instance_sizes" {  
  type = map(string)  
  description = "Instance sizes to use in AWS »  
  default = {  
    small = "t2.micro"  
    medium = "t2.small"  
    large = "t2.large"  
  }  
}
```

Récupérer une valeur d'une variable de type « map »

```
var.aws_instance_sizes.small    OU    var.aws_instance_sizes["small"]
```

Utilisation des Inputs et Outputs

Améliorations – Modification du code

Exemple :

```
variable "aws_region" {  
  description = "La région AWS dans laquelle déployer les ressources"  
  type        = string  
  default     = "us-east-1"  
}
```

Utilisation des Inputs et Outputs

Améliorations – Modification du code (exercice)

- # Bloc CIDR VPC
- # Activer les noms d'hôte DNS
- # Bloc CIDR de sous-réseau
- # Mapper l'adresse IP publique au lancement
- # Numéro de port d'entrée – aucune valeur par défaut
- # Type d'instance – aucune valeur par défaut

Utilisation des Inputs et Outputs

Valeurs de sortie (Output values)

Dans Terraform, les **valeurs de sortie (Outputs)** sont des variables **déclarées dans vos fichiers .tf** qui permettent de **recupérer et afficher des informations importantes** après l'exécution d'un plan ou d'un apply. Elles sont très utiles pour plusieurs raisons.

1. Communiquer des informations importantes

Les outputs permettent de **montrer des valeurs générées automatiquement** par Terraform, comme :

- l'IP publique d'une instance EC2
- l'ID d'une VPC ou d'un subnet

2. Faciliter l'automatisation / pipelines CI/CD

Les outputs peuvent être lus par des outils externes pour automatiser :

- la configuration d'un serveur
- le déploiement d'une application
- l'intégration dans un script

Utilisation des Inputs et Outputs

Syntaxe - Output values

```
output "name_label" {  
  value      = value  
  description = "string"  
}
```

Exemple :

```
output "aws_instance_public_dns" {  
  description = "Nom d'hôte DNS public de l'instance EC2"  
  value       = aws_instance.nginx1.public_dns  
}
```

Utilisation des Inputs et Outputs

Améliorations – Communiquer des informations pertinentes (exercice)

ID du VPC créé

ID du sous-réseau public

Utilisation des Inputs et Outputs

Formater et valider le code Terraform

```
terraform fmt
```

Cette commande sert à **formater automatiquement vos fichiers Terraform** (*.tf, *.tfvars) selon les **règles officielles de style** de HashiCorp.

```
terraform fmt -check
```

Avec cette option, la commande fait **la même analyse**, mais **sans modifier les fichiers**.
=> Elle affiche les fichiers mal formatés, mais ne les change pas.

Utilisation des Inputs et Outputs

Formater et valider le code Terraform

```
terraform validate
```

Cette commande sert à **valider la syntaxe et la cohérence du code Terraform**.

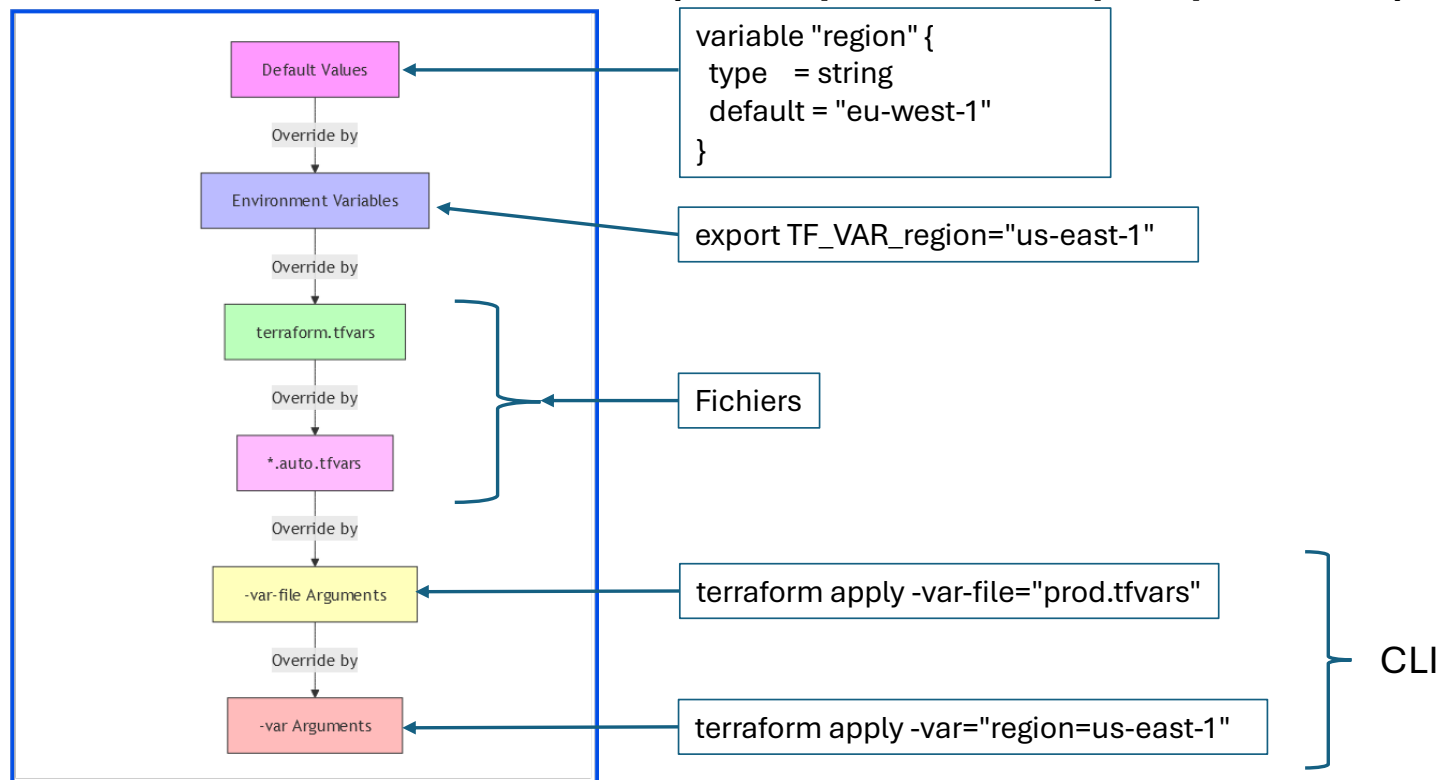
En d'autres termes, elle vérifie :

- que les fichiers `.tf` sont **syntactiquement corrects**
- que les types de variables sont compatibles
- que les blocs Terraform sont correctement structurés
- que les variables sont bien déclarées

```
> terraform validate  
Success! The configuration is valid.
```

Utilisation des Inputs et Outputs

Fournir des valeurs aux variables (moins prioritaire au plus prioritaire)



Utilisation des Inputs et Outputs

Fournir des valeurs aux variables (moins prioritaire au plus prioritaire)

```
terraform.tfvars > ...  
1 http_port          = 80  
2 ec2_instance_type = "t2.micro"
```

Après exécution de terraform apply :

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

```
aws_instance_public_dns = "ec2-3-236-41-97.compute-1.amazonaws.com"  
public_subnet_id        = "subnet-06029ce0bfae3dcb6"  
vpc_id                   = "vpc-0a8b7a954b617a009"
```

Valeur de sortie (Outputs)

Utilisation des expressions et fonctions

Améliorations

- Mettre en place des tags par défaut pour les ressources AWS et des conventions de nommage par défaut basées sur l'environnement et le projet.

```
+ owner_id          = {  
+ tags              = {  
  + "Company"       = "HeH"  
  + "Environment"   = "dev"  
  + "Name"          = "web_app-dev-vpc"  
  + "Project"       = "web_app"
```

- Dissocier le script de démarrage des fichiers de configuration et le stocker dans son propre fichier pour pouvoir le mettre à jour et le réutiliser.
- il serait bien que la sortie DNS publique soit un lien cliquable et pas seulement le nom DNS.

Outputs:

[Follow link](#) (cmd + click)

```
aws_instance_public_dns = "http://ec2-3-234-209-204.compute-1.amazonaws.com:80"  
public_subnet_id = "subnet-0216935072081bc39"  
vpc_id = "vpc-0df31412eea7df40f"
```


Utilisation des expressions et fonctions

Valeurs locales

Les **valeurs locales** dans Terraform servent à définir des **variables internes** réutilisables dans votre configuration, sans être des variables d'entrée.

Elles rendent votre code **plus clair et plus maintenable**.

Définition

```
locals {  
  instance_prefix = »heh"  
  common_tags = {  
    company    = »heh_DST"  
    project    = var.project  
  }  
}
```

Utilisation

```
local.instance_prefix  
local.common_tags.company
```

Utilisation des expressions et fonctions

Tags AWS

Un tag AWS est une **étiquette** ou un libellé qui peut être **attaché à une ressource spécifique**, telle qu'une instance EC2, une base de données RDS, un bucket S3 ou tout autre élément de notre infrastructure cloud.

Chaque tag se compose de deux parties :

- une clé (key) : est un nom que vous attribuez au tag.
- une valeur (value) : est une description ou une caractéristique spécifique associée à cette clé.

Vous pourriez avoir un **tag** avec la **clé "Environnement"** et la **valeur "Développement"** pour indiquer que la ressource est utilisée dans un environnement de développement.

Les tags sont des informations additionnelles que nous pouvons ajouter pour organiser, classer et gérer nos ressources.

Utilisation des expressions et fonctions

Tags AWS - Syntaxe

```
resource "aws_instance" "web" {  
  ami      = "ami-123456"  
  instance_type = "t2.micro"  
  
  tags = {  
    Company    = "HeH"  
    Environment = "dev"  
    Name       = »web_app-dev-ec2"  
    Project    = »web_app"  
  }  
}
```

Utilisation des expressions et fonctions

Tags AWS - Exemple

OBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
+ resource "aws_vpc" "app" {
  + arn                                = (known after apply)
  + cidr_block                        = "10.0.0.0/16"
  + default_network_acl_id           = (known after apply)
  + default_route_table_id           = (known after apply)
  + default_security_group_id        = (known after apply)
  + dhcp_options_id                  = (known after apply)
  + enable_dns_hostnames              = true
  + enable_dns_support               = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                               = (known after apply)
  + instance_tenancy                  = "default"
  + ipv6_association_id               = (known after apply)
  + ipv6_cidr_block                   = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id               = (known after apply)
  + owner_id                          = (known after apply)
  + tags                              = {
    + "Company"      = "HeH"
    + "Environment"  = "dev"
    + "Name"         = "web_app-dev-vpc"
    + "Project"      = "web_app"
  }
  + tags_all = {
```

} Tagger la ressource VPC

Utilisation des expressions et fonctions

Tags AWS - Avantages

En associant des tags cohérents à chaque ressource :

- On identifie rapidement à quelle application, projet ou équipe appartient chaque élément.
- Cela permet de suivre précisément les dépenses cloud.
- On peut les utiliser pour déclencher des actions spécifiques ou des politiques de gestion (automatisation).
Par exemple, créons une politique qui met automatiquement hors service les instances EC2 portant le tag "Environnement" avec la valeur "Développement" en dehors des heures de bureau pour diminuer les coûts.

Utilisation des expressions et fonctions

Tags AWS - Exemple

variables.tf > ...

```
42  
43 variable "company_name" {  
44     description = "Le nom de la société"  
45     type        = string  
46     default     = "HeH"  
47 }  
48  
49 variable "project" {  
50     description = "Le nom du projet"  
51     type        = string  
52 }  
53  
54 variable "environment" {  
55     description = "L'environnement pour le déploiement(e.g., dev,  
56     type        = string  
57 }
```

Définition des variables

Utilisation des expressions et fonctions

Tags AWS - Exemple

```
locals.tf > ...  
1 locals {  
2   common_tags = {  
3     Company    = var.company_name  
4     Project     = var.project  
5     Environment = var.environment  
6   }  
7   naming_prefix = "${var.project}-${var.environment}"  
8 }
```

Définition de la valeur locale

Local.common_tags : permet d'avoir des tags standard sur toutes les ressources.

Utilisation des expressions et fonctions

Tags AWS - Exemple

```
44 resource "aws_internet_gateway" "app" {
45     vpc_id = aws_vpc.app.id
46     tags   = local.common_tags
47 }
```

```
36 # NETWORKING #
37 resource "aws_vpc" "app" {
38     cidr_block          = var.vpc_cidr_block
39     enable_dns_hostnames = var.vpc_enable_dns_hostnames
40
41     tags = merge(local.common_tags, { Name = lower("${local.naming_prefix}-vpc") })
42 }
```

Ce tag **Name** devient le nom logique du VPC dans la console AWS.

La fonction merge() de Terraform **fusionne** deux maps de tags

Utilisation des expressions et fonctions

Tags AWS - Exemple

```
terraform.tfvars ×  
terraform.tfvars > ...  
1 http_port          = 80  
2 ec2_instance_type = "t2.micro"  
3  
4 project            = "web_app"  
5 environment        = "dev"
```

Utilisation des expressions et fonctions

Expressions Terraform

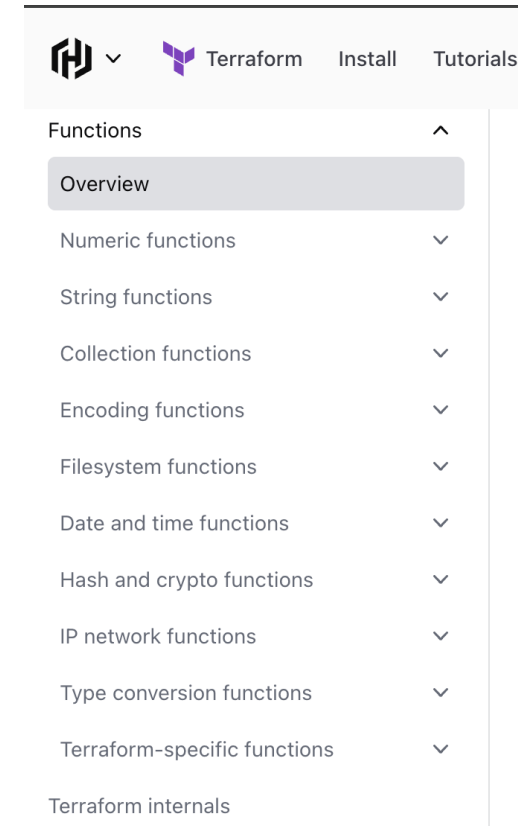
- Expressions littérales
 - **Interpolation de chaîne** : Permet de construire des chaînes dynamiques
`${var.project}-${var.environment}`
- Références d'objet ou d'attribut (Les références permettent d'utiliser des valeurs définies ailleurs)
(référence à une variable : `var.environment`, référence à une ressource : `aws_instance.web.id`, ...)
- Opérateurs arithmétiques et logiques
- Expressions conditionnelles
- Expressions de collections (for)
 - permet de manipuler des **listes**, **maps**, **sets** avec des expressions avancées.

Utilisation des expressions et fonctions

Fonctions

Terraform possède **de nombreuses fonctions intégrées** qui permettent de transformer des données, manipuler des listes/maps, travailler avec les réseaux, les chaînes, l'encodage, etc.

Elles sont organisées par catégories dans la documentation en ligne.



Utilisation des expressions et fonctions

Console Terraform – Tester les expressions et les fonctions

```
terraform console
```

C'est un outil **interactif** intégré à Terraform qui permet d'évaluer des **expressions Terraform en direct**, exactement comme Terraform les traiterai pendant un `plan` ou un `apply`.

Terraform charge automatiquement :

- variables (si définies ou dans un `.tfvars`)
- locals
- données statiques
- l'état (`terraform.tfstate`) si existe

Utilisation des expressions et fonctions

Console Terraform – Tester les expressions et les fonctions

terraform console est indispensable pour :

- comprendre pourquoi un plan ne donne pas le résultat attendu
- débbugger un nom, un tag, un CIDR, une interpolation
- tester des expressions complexes **avant** de les mettre dans du code
- vérifier les ressources dans l'état sans ouvrir le `.tfstate` à la main

```
> terraform console
> lower("HEH")
"heh"
> local.common_tags
{
  "Company" = "HeH"
  "Environment" = "dev"
  "Project" = "web_app"
}
> merge(local.common_tags, { Name = lower("${local.naming_prefix}-vpc") })
{
  "Company" = "HeH"
  "Environment" = "dev"
  "Name" = "web_app-dev-vpc"
  "Project" = "web_app"
}
> exit
```

`base_web_app` on `main [!]` via `default` on `(us-east-1)` took **32s**

Utilisation des expressions et fonctions

Fonctions – Templatefile

La fonction **templatefile()** dans Terraform est extrêmement utile pour **générer dynamiquement du texte** ou des fichiers à partir d'un **template externe** et de variables. C'est souvent utilisé pour les scripts, fichiers de configuration ou User Data pour EC2.

`templatefile()` lit un fichier externe et remplace les **placeholders** par les valeurs fournies.

```
templatefile(file_path, { map of variables })
```

- **file_path** : chemin vers le fichier template (ex : "startup_script.tpl")
- **{map of variables}** : map des variables à injecter dans le template

Utilisation des expressions et fonctions

Fonctions – Templatefile

Le fichier template utilise la **syntax HCL de Terraform** pour les placeholders :

```
#!/bin/bash  
echo "Hello ${name}"  
echo "Environment: ${environment}"
```

`${name}` et `${environment}` seront remplacés par les valeurs fournies dans la map.